

Lab 13: Implement String Editing Algorithm

Theory:

The String Editing (Edit Distance) algorithm finds the minimum number of operations (insert, remove, replace) required to transform one string into another. It is a classic dynamic programming problem.

Complexity Overview

Resource	Complexity	Condition
Time	$O(m * n)$	m and n are string lengths
Space	$O(m * n)$	2D DP Table storage

Algorithm

1. Create a 2D DP array of size $(m+1) \times (n+1)$.
2. Initialize the first row and column.
3. Iterate over the strings. If characters match, copy the diagonal element.
4. If they don't match, take the minimum of left, top, and diagonal values and add 1.

Pseudocode

```
BEGIN
  FOR i ← 0 to m DO
    FOR j ← 0 to n DO
      IF i = 0 THEN
        dp[i][j] ← j
      ELSE IF j = 0 THEN
        dp[i][j] ← i
      ELSE IF str1[i-1] = str2[j-1] THEN
        dp[i][j] ← dp[i-1][j-1]
      ELSE
        dp[i][j] ← 1 + min(dp[i][j-1], dp[i-1][j], dp[i-1][j-1])
      END IF
    END FOR
  END FOR
  RETURN dp[m][n]
END
```

Conclusion:

The simulation systematically solves the Edit Distance sub-problems bounded by an $(m+1) \times (n+1)$ Dynamic Programming table. With input vectors mapped dynamically through character iteration, overlapping sub-comparisons evaluate constraints algorithmically preventing duplicate structural computation.

By tracking back operations necessary to transpose the baseline string "sunday" into the target string "saturday", the calculated matrix validates exactly 3 optimal operations. These sequential modifications (insertion of 'a', insertion of 't', substitution of 'n' for 'r') accurately demonstrate the robust and precise transposition nature of the String Editing (Levenshtein Distance) algorithm.